

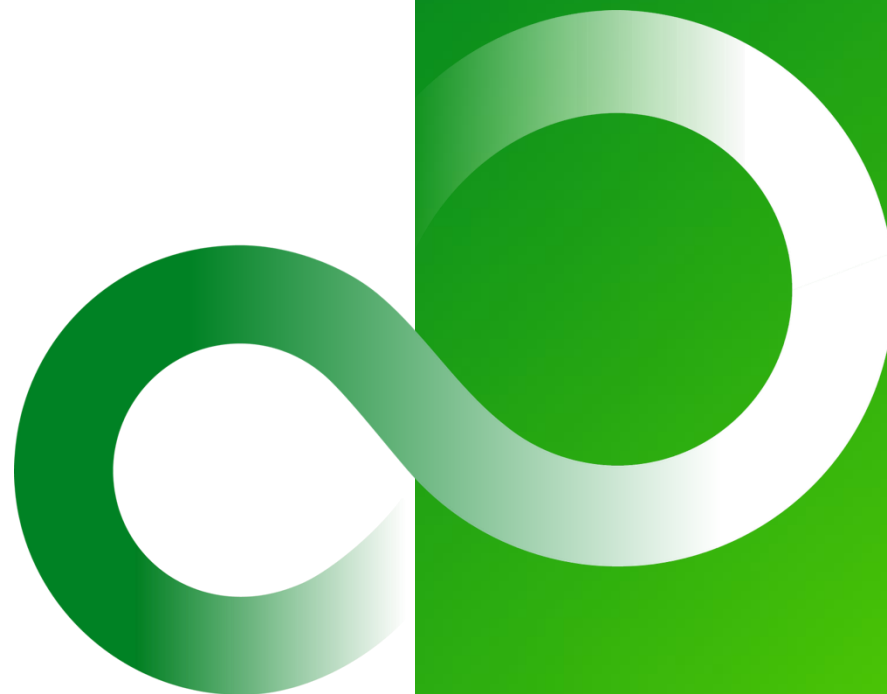
GitHub Enterpriseの 生成AI機能を活用した 開発スタイル変革への挑戦

2026/09/17

富士通株式会社

グローバルデリバリーBG

ソフトウェアオープンイノベーション事業本部





吉田 学

he/him

マネージャー

グローバルデリバリーBG

ソフトウェアオープンイノベーション事業本部

連絡先



044-331-1111



yoshidam@fujitsu.com



Fujitsu Uvance
Kawasaki Tower

担当領域

- 富士通のアプリケーションフレームワーク「INTARFRM」の開発責任者として、製品ロードマップ策定と開発チームを牽引。
加えて、LLMを活用した脆弱性検知・修正の**社内テックリード**を担い、セキュリティと開発生産性の両立に向けたAI活用を推進している。

キャリア・実績

- FY2026 : LLMを活用した脆弱性検知・修正の社内テックリード。
- FY2025 : GitHub Copilotを製品開発・保守へ導入。生産性向上に寄与。
- FY2023～ : 本部の生成AI WGに参画し、活用事例などを全社へ発信、普及に寄与。

趣味・休日の過ごし方

- 趣味はスポーツ観戦、読書、ウォーキングです。
- 休日には、たまにフットサルに行きます。

「人の実装」から「AI委譲+レビュー」へ

○背景

- 生成AIは、コード補助からタスクを委譲できるAIエージェントへ進化
- 製品保守では、脆弱性対応・機能強化のスピードと品質の両立が必須
- ★人の稼働時間に依存しない非同期実行と、人によるレビュー・承認を組み合わせる

○目的

- 実業務におけるAIエージェント活用の実現性・効果を、リードタイム、作業・確認負荷、リソース制約の観点で評価

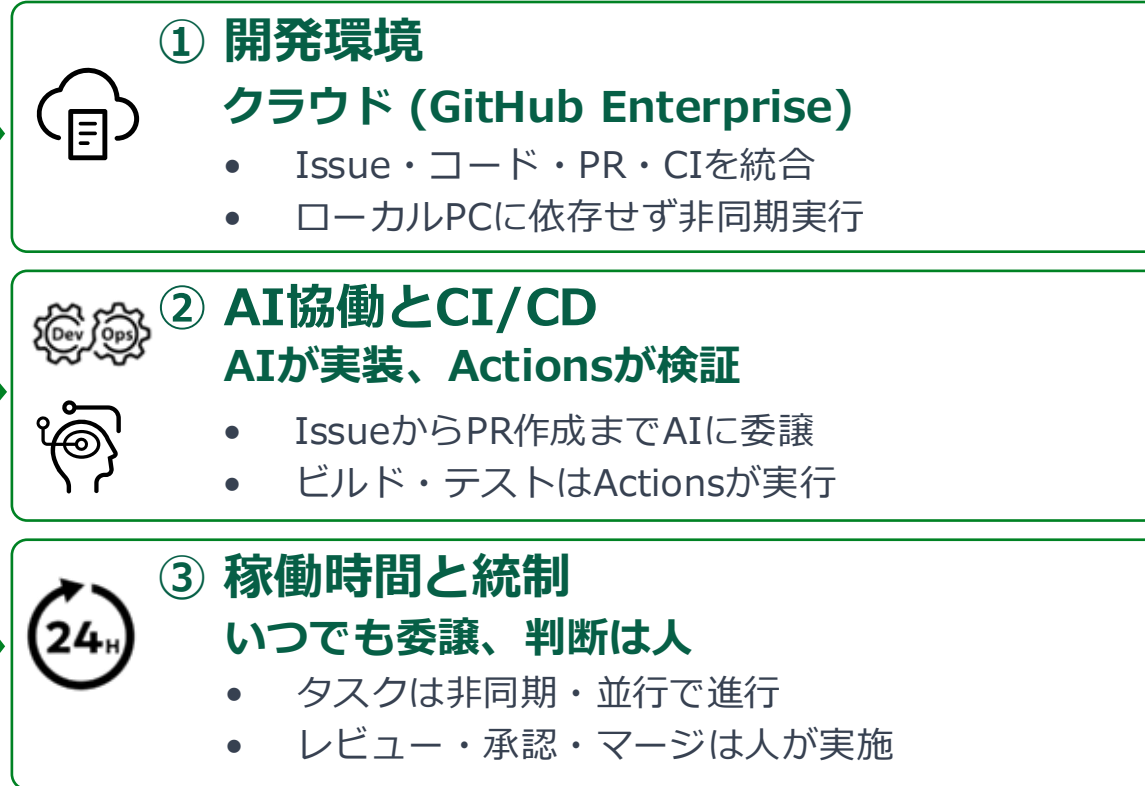
○検証概要

- 保守業務に適用し、実業務ベースで効果を評価
- GitHub Enterprise (GHE) 中心フローと既存のAzure DevOps (ADO) 中心フローを同一条件で評価

AS-IS : ADO中心（2026年4月検証時点）



To-Be : GHE+AIエージェント



To-Beの特徴

 **非同期クラウド実行**
任意の時間にタスクを委譲。
ローカルPCに依存せず実行

 **明確な役割分担**
AIが実装、Actionsが検証。
人がレビュー・承認・マージ

 **統合と統制**
Issue・コード・PR・CIを統合。
リリース判断は人が担う

GitHub Enterprise × GitHub Copilot検証



和泉 肇弥

一般社員

グローバルデリバリーBG

ソフトウェアオープンイノベーション事業本部

連絡先



044-331-1111



izumi.toshiya@fujitsu.com



**Fujitsu Uvance
Kawasaki Tower**

担当領域

- AI関連技術検証、製品開発保守担当。
Spring Boot、Angular、Vueを用いたWebアプリケーション開発に従事。最近は、バ
イブコーディングと仕様駆動開発を組み合わせたAI活用型のWebアプリ開発に取り組
んでいるほか、VS Code拡張機能の開発も行っている。

キャリア・実績

- FY2026 : ハーネスの方式標準化。LLMを活用した便利拡張機能開発。
- FY2025 : GitHub Copilotを開発全工程で活用。生産性向上に寄与。
- FY2023～ : 本部の生成AI WGに参画し、活用事例などを全社へ発信、普及に寄与。

趣味・最近の出来事

- ゲーム：対戦（DbD）、パズル（Baba Is You）、RPG（オクトラ2）
- 運動不足解消のために最近ジム（プール）に通い始めたが外耳炎になり中断中

○ 概要

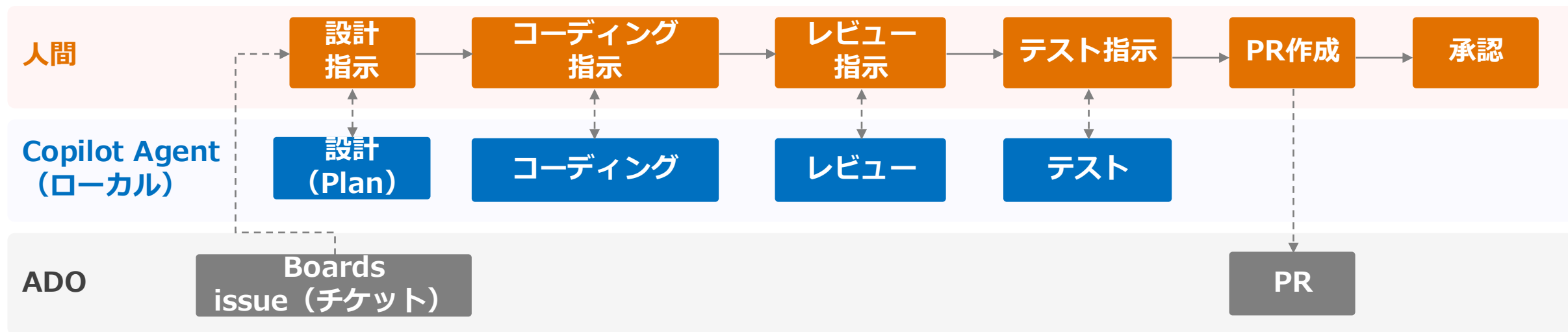
- 既に完了した修正をGHE×GitHub Copilotで再度修正し
完了したアウトプットとの差分を定量・定性評価

○ 検証対象

製品	規模	主な技術スタック	今回の対象
製品A	100 Kstep	Vue / TypeScript / Java	本検証について紹介
製品B	500 Kstep	JSP / Java	

○ 検証期間

- 2026/2 - 2026/3（2 か月間）
※今回の共有は、上記検証期間中の内容となるため、一部情報については古い場合があります。



—— 手順
----- 連携

課題

人間の指示が常に必要

AIは各工程を支援できるが、起動・指示・判断のトリガーがすべて人間。工程間の自動化ができていない

ヒューマンエラー

手動作業により抜け漏れが起きる
(テスト実行忘れ、レビュー漏れ等)

リソース拘束

1件ずつローカルで対応するため、人間もマシンリソースも拘束される

GHE + GitHub Copilotで利用した主な機能

✓ : 今回検証した機能

① Copilot Coding Agent ✓

IssueをアサインするだけでAIがクラウド上でコード生成 → Branch → PR作成まで自律実行

② Copilot Code Review ✓

PR作成時にAIがコード品質・整合性を自動チェックし、指摘コメントをPRに投稿

③ @copilot メンション ✓

Issue・PRのコメント欄で @copilot ○○して と書くだけでエージェントが応答・追加修正を実行

④ MCP対応 ✓

Playwright / DB / 各種SaaS等の外部ツールを、AIEージェントから直接呼び出せる拡張の仕組み

⑤ Copilot PR Summary

PRの差分をAIが解析し、変更概要・レビュー観点を含む説明文を自動生成

⑥ Copilot Autofix ✓

CodeQL で検出したセキュリティ脆弱性の修正案をAIが自動生成・PRへ提案

これらを GitHub Actions (CI/CDワークフロー) と組み合わせることで、**CI失敗時の自動修正ループ**など、実装・検証を自動化し、人がレビュー・承認するフローを構築可能 (本検証の主対象)

GHEを前提とした開発フロー想定



○ 期待

工程間の自動化・納期短縮

複数Issueを自動で同時並行に処理することで、人間の指示を待つことなく完了し、納期を短縮できる

ヒューマンエラー防止

ワークフローで定義した手順が必ず実行される。テスト忘れ・レビュー漏れを抑制

リソースの解放

設計判断・アーキテクチャ検討など、より複雑な作業に集中できる

○調査内容

調査	対象件数	比較内容
①起票～レビュー完了	14件 軽微な障害・改善に限定	Issue起票からレビュー完了までのリードタイム
②テスト実行～承認	6 件	テスト実行から修正、再テスト完了までのリードタイム

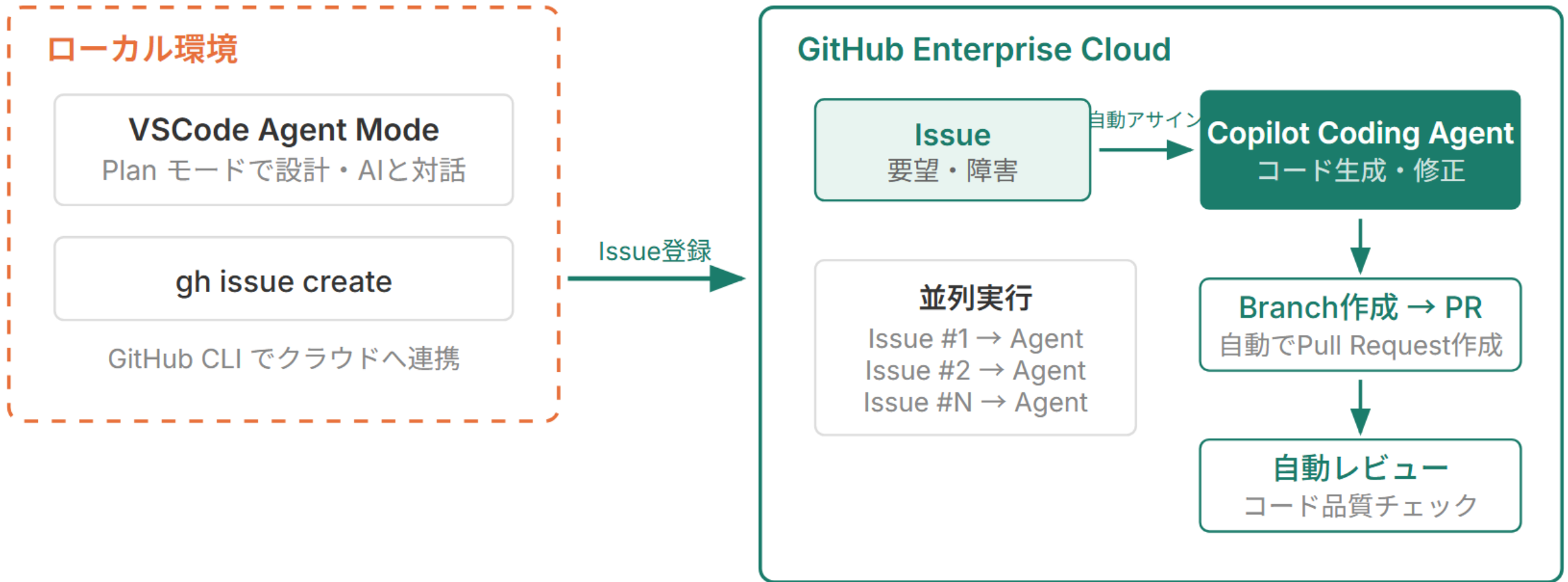
検証環境

- ・ エディタ : VSCode
- ・ エージェント : GitHub Copilot Agent
- ・ LLM : Claude Opus 4.6

対象Issueの特徴と検証完了条件

- 既に修正済みの小規模な障害・機能改善が対象。
検証完了条件は以下
- ・ 既存修正とGHE上での修正結果が同等レベルである
 - ・ UT/E2Eテストが正常完了し動作が正しいこと

検証①：起票～レビュー完了



特別な設定や構築は不要で標準機能のみで実現可能。
設計フェーズはGHE上で実行することも可能だが、ローカルとの機能的差異はないため、使い慣れたローカルで実施。

検証結果：リードタイム比較（14件）

○1件当たりのリードタイム（バー＝平均 / 下の線＝最小～最大）



○14件を同時開発した場合の完了までの経過時間（逐次 vs 並列）



1件当たりはADOの方が早いですが、並列実行により件数が増えるほどGHEが有利。
人間の实働がほぼゼロになる点が最大の価値

検証② : テスト自動化～自動修正

検証時のアーキテクチャ

GitHub Enterprise Cloud

Pull Request
push / open

GitHub Actions Runner

Unit Tests

Node.js 20 / npm ci

lint

→ vitest

→ coverage

結果出力

E2E Tests

WAR取得

→ ビルド

Docker

Payara

PgSQL

↓ デプロイ

Playwright

結果出力

レポート → Artifacts 保存

自動修正ループ

テスト成功

→ レビュー・マージへ

テスト失敗

1. PRにコメント投稿
@copilot + 失敗情報

Copilot Coding Agent

2. 修正 → push → 再テスト

技術補足： CIから @copilot をそのままメンションすると、コメントの投稿者が Copilot（GitHubのシステムユーザー）扱いになり、@copilotが反応しない。エージェントを起動するにはアクセス権のあるメンバーからのメンションである必要があるため、PAT（Personal Access Token）を使ってメンバーとしてコメントを投稿するようにした。

検証結果：リードタイム比較（6件）

○ 1件当たりのリードタイム

ADO

平均 30 分

GHE

平均 23 分

実働：ほぼゼロ、6件すべて自動修正成功

- ・ e2eテストは修正/実行共にコストが大きく、ローカル実行は他作業を妨げるため
GHE上で完結するメリットが特に大きい
- ・ 環境構築トラブルが排除され、テスト実行忘れ等のヒューマンエラーを抑制

⚠ **課題** Actionsのワークフロー実行許可を手動承認する必要があり、自動修正ループのたびに承認操作が発生する。
自動化と承認頻度の両立が課題。

評価・まとめ

○ 評価内容

- GHEをフル活用した場合の開発期間をADOの開発と比較し、リードタイム短縮効果を試算想定
- 開発概要：OS・ミドルウェア更新、要望・障害対応、機能追加

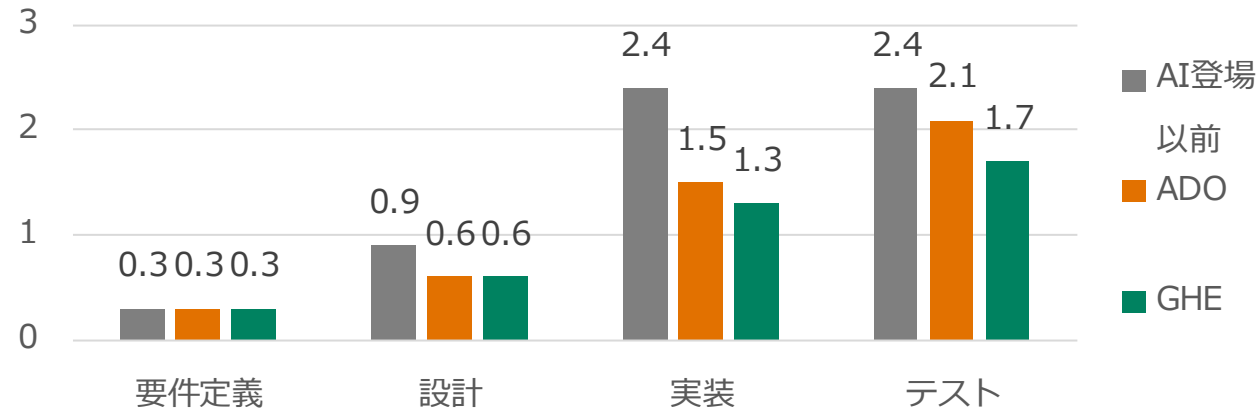
○ 条件

- 従来開発のAI効果を社内実績より、開発全体の効率化を25%と設定
- 検証結果を踏まえ、GHEの効果を35%（+約10%）と試算
- 保守業務は6か月に1回のリリースサイクル

開発期間の短縮



工程別の工数（人月）



結論 GHE活用により開発期間を約35%短縮（6ヵ月→3.9ヵ月）し、リリース頻度を年2回→年3回へ向上可能

○当初の期待

工程間の自動化・納期短縮

複数Issueを自動で同時並行に処理することで、人間の指示を待つことなく完了し、納期を短縮できる

ヒューマンエラー防止

ワークフローで定義した手順が必ず実行される。テスト忘れ・レビュー漏れを抑制

リソースの解放

設計判断・アーキテクチャ検討など、より複雑な作業に集中できる

○評価

◎ 期待通り

並列実行や自動修正を活用することでAI導入前比の短縮率が25%から35%へ向上（+10pt）

◎ 期待通り

ワークフローにより実現
ヒューマンエラーが抑制された

○ 概ね期待通り

実働ほぼゼロ化を達成
ただしワークフロー承認の自動承認と手動承認の頻度について課題あり

○結論

- 本検証条件ではGHE中心フローが有望
- 開発期間の短縮：**従来25% → GHE活用35%**（+10pt）
- リリース頻度：**年2回 → 年3回への向上**が見込める
- 全体最適の観点でGHE（並列・自動化）が有効

○今回の検証の立ち位置と今後

- 今回の検証では「**軽微な修正はGHE、複雑な対応はローカル**」のハイブリッド運用を実施
- 中期的にはGHE中心の開発へ段階的にシフト（開発基盤の標準化）

○課題

- AIエージェントによる自律開発の実現（自律性向上／ハーネスなど）
- 開発プロセスの再設計や品質担保の仕組み整備が必須

Thank you

